

# **Tutorium 9**

## Altklausuraufgaben: Bäume

Grundlagen der Praktischen Informatik

15. Juni 2026

Zur Erinnerung: Ein Baum und die foldTree Funktion.

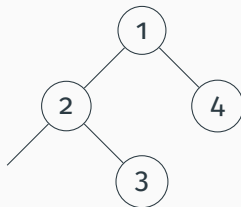
## Definition

```
1  data BinTree a = Node a (BinTree a) (BinTree a) | Empty
2      deriving (Eq, Show)
3
4  foldTree :: (a -> b -> b -> b) -> b -> BinTree a -> b
5  foldTree fNode fEmpty = fold
6      where
7          fold (Node a l r) = fNode a (fold l) (fold r)
8          fold Empty = fEmpty
```

# Beispielbaum aus der Altklausur

## bsp Baum

```
1 bsp = Node 1 (Node 2 Empty (Node 3 Empty Empty))  
2       (Node 4 Empty Empty)
```



# Aufgabe 4a: Innere Knoten zählen

**Aufgabe:** `countInnerNodes` zählt die Anzahl der inneren Knoten (Knoten, die kein Blatt sind).

## Rekursive Lösung

```
1 countInnerNodes' :: BinTree a -> Int
2 countInnerNodes' Empty = 0
3 countInnerNodes' (Node _ Empty Empty) = 0
4 countInnerNodes' (Node _ l r) =
5     1 + countInnerNodes' l + countInnerNodes' r
```

# Aufgabe 4b: Innere Knoten auflisten

**Aufgabe:** `listInnerNodes` listet die Werte aller inneren Knoten in einer Liste auf.

## Rekursive Lösung

```
1 listInnerNodes' :: BinTree a -> [a]
2 listInnerNodes' Empty = []
3 listInnerNodes' (Node _ Empty Empty) = []
4 listInnerNodes' (Node a l r) =
5     a : (listInnerNodes' l ++ listInnerNodes' r)
```

# Aufgabe 4c: Baum-Produkt mit foldTree

**Aufgabe:** `productTree` berechnet das Produkt aller Werte mit `foldTree`.

## Vergleich: Rekursion vs. Fold

```
1  -- Rekursiv:
2  productTree' Empty = 1
3  productTree' (Node a l r) = a * productTree' l * productTree' r
4
5  -- Mit foldTree:
6  productTree'' :: Num a => BinTree a -> a
7  productTree'' = foldTree (\a l r -> a * l * r) 1
```

# Live Demo



01\_altklausuraufgaben\_baeume.hs

Wir besprechen diese Klausuraufgaben zusammen in der IDE!